Data Warehouses

Lecture 11

Lecture Goals

Some notes on building cubes in SSAS

Two approaches

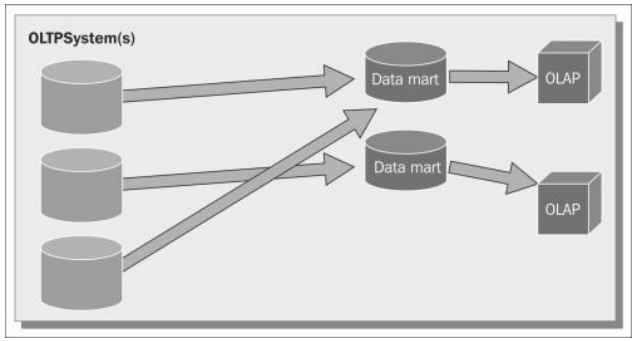
They are described in the books of the following two leading authors:

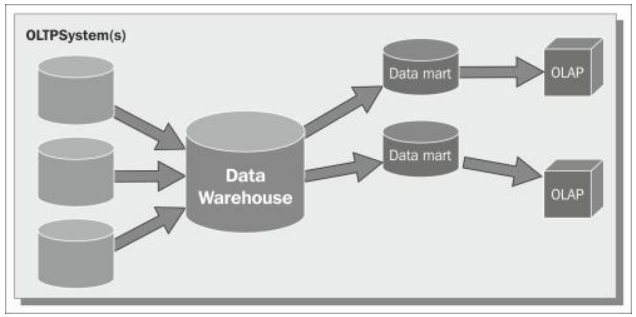
Ralph Kimball:

If we are building a Kimball data warehouse, we build fact tables and dimension tables structured as data marts. We will end up with a data warehouse composed of the sum of all the data marts.

Bill Inmon:

If our choice is that of an Inmon data warehouse, then we design a (somewhat normalized) physical relational database that will hold the data warehouse. Afterwards, we produce departmental data marts with their star schemas populated from that relational database.





Start with a data mart

Whether you are using the Kimball or Inmon methodology, the frontend database just before the Analysis Services cube should be a data mart.

- A data mart is a database that is modeled according to the rules of Kimball's dimensional modeling methodology and is composed of fact tables and dimension tables.

Apart from the issue of modeling data in an appropriate way for Analysis Services, it's also important to understand how details of the physical implementation of the relational data mart can be significant too.

- All of the dimension and fact tables you intend to use should exist within the same relational data source
  - *for example, if you're using SQL Server, this means all the tables involved should exist within the same SQL Server database*

Proper data types

When we design data marts, we need to be aware that Analysis Services does not treat all data types the same way.

The cube will be much faster, for both processing and querying, if we use the right data type for each column.

Fact column type	Fastest SQL Server data types
Surrogate keys	`tinyint`, `smallint`, `int`, and `bigint`
Date key	`int` in the format yyyymmdd
Integer measures	`tinyint`, `smallint`, `int`, and `bigint`
Numeric measures	`smallmoney`, `money`, `real`, and `float` (Note that decimal and vardecimal require more CPU power to process than money and float types)
Distinct count columns	`tinyint`, `smallint`, `int`, and `bigint` (If your count column is `char`, consider either hashing or replacing with surrogate key)

When Analysis Services needs to process a cube or a dimension, it sends queries to the relational database in order to retrieve the information it needs.

- Dimension processing
  - *During dimension processing, Analysis Services sends several queries, one for each attribute of the dimension*
  - *In the form of SELECT DISTINCT ColName, where ColName is the name of the column holding the attribute.*
    - Many of these queries are run in parallel
- Dimensions with joined tables
  - *If a dimension contains attributes that come from a joined table, the JOIN is performed by SQL Server, not Analysis Services.*
    - Analysis Services will send a query to SQL Server containing the INNER JOIN between the two tables.
    - This situation arises very frequently when we define snowflakes instead of simpler star schemas.
- Fact dimensions
  - *The processing of dimensions related to measure group with a fact relationship type, usually created to hold degenerate dimensions, is performed in the same way as any other dimension.*
    - This means that SELECT DISTINCT will be issued on all the degenerate dimension's attributes.
    - Clearly, as the dimension and the fact tables are the same, the query will ask for DISTINCT over a fact table

Continued

Distinct Count measures

The last kind of query that we need to be aware of is when we have a measure group containing a DISTINCT COUNT measure.

In this case, due to the way Analysis Services calculates distinct counts, the query to the fact table will be issued with ORDER BY for the column we are performing the distinct count on.

Needless to say, this will lead to very poor performance because we are asking SQL Server to sort a fact table on a column that is not part of the clustered index

(usually, the clustered index is built on the primary key)

There are some optimizations, mostly pertinent to partitioning, that need to be done when we have DISTINCT COUNT measures in very big fact tables.

Continued

- Indexes in the data mart
  - *The usage of indexes in data mart is a very complex topic and we cannot cover it all in a simple section.*
  - *Nevertheless, there are a few general rules that can be followed for both fact and dimension tables.*
    - Dimension tables should have a primary clustered key based on an integer field, which is the surrogate key.
      - Non-clustered indexes may be added for the natural key
        - *in order to speed up the ETL phase for Slowly Changing Dimensions.*
    - It is questionable whether fact tables should have a primary key or not.
      - to have a primary clustered key based on an integer field, because it makes it very simple to identify a row in the case where we need to check for its value or update it.
        - *In the case where the table is partitioned by date, the primary key will be composed of the date and the integer surrogate key, to be able to meet the needs of partitioning.*
      - If a column is used to create a DISTINCT COUNT measure in a cube, then it might be useful to have that column in the clustered index

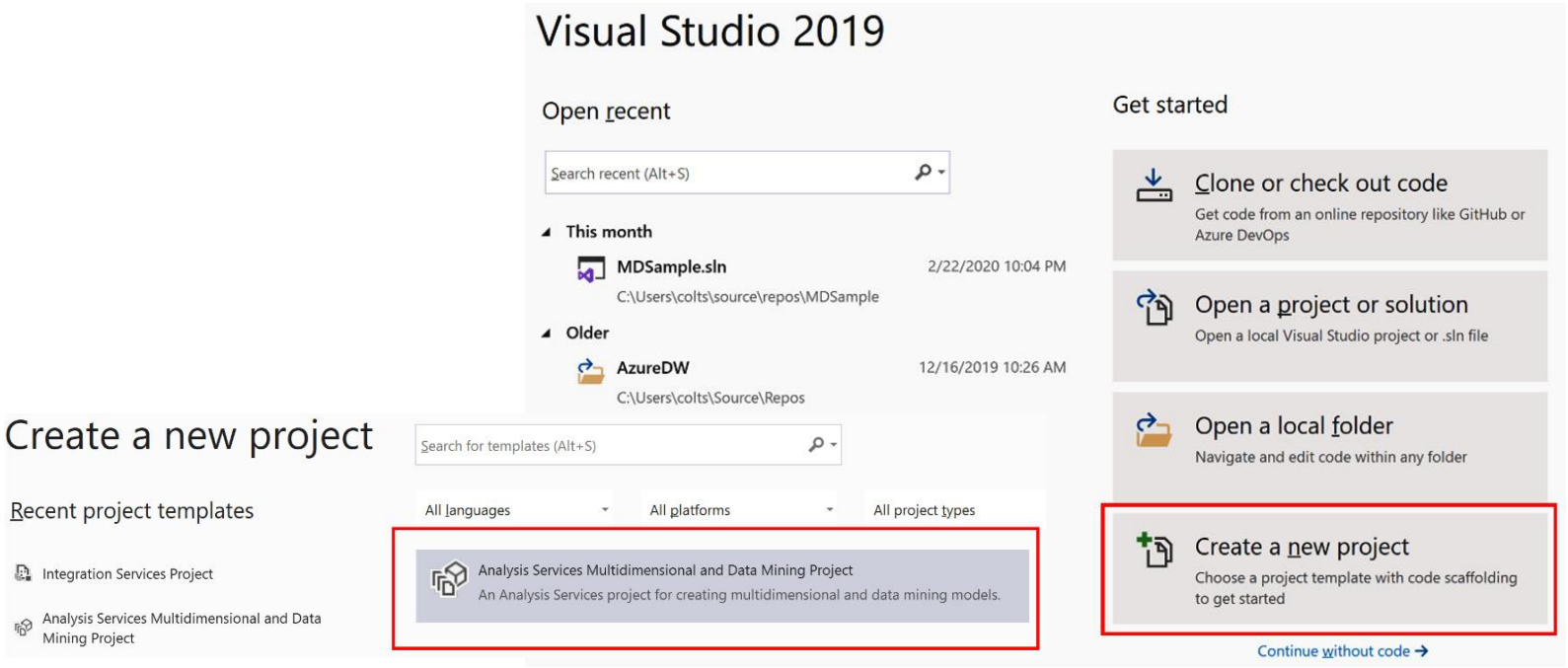
Continued

- Once the cube has been built:
  - *if MOLAP storage is being used*
    - no other indexes are useful.
  - *if the data mart is queried by other tools such as Reporting Services, or if ROLAP partitions are created in Analysis Services*
    - then it might be necessary to add more indexes to the tables.
- Remember
  - *that indexes slow down update and insert operations so they should be added with care.*
  - *A deep analysis of the queries sent to the relational database will help to determine the best indexes to create.*

Let’s start

At first, you need to create a new Analysis Services project in Visual Studio (SSDT)

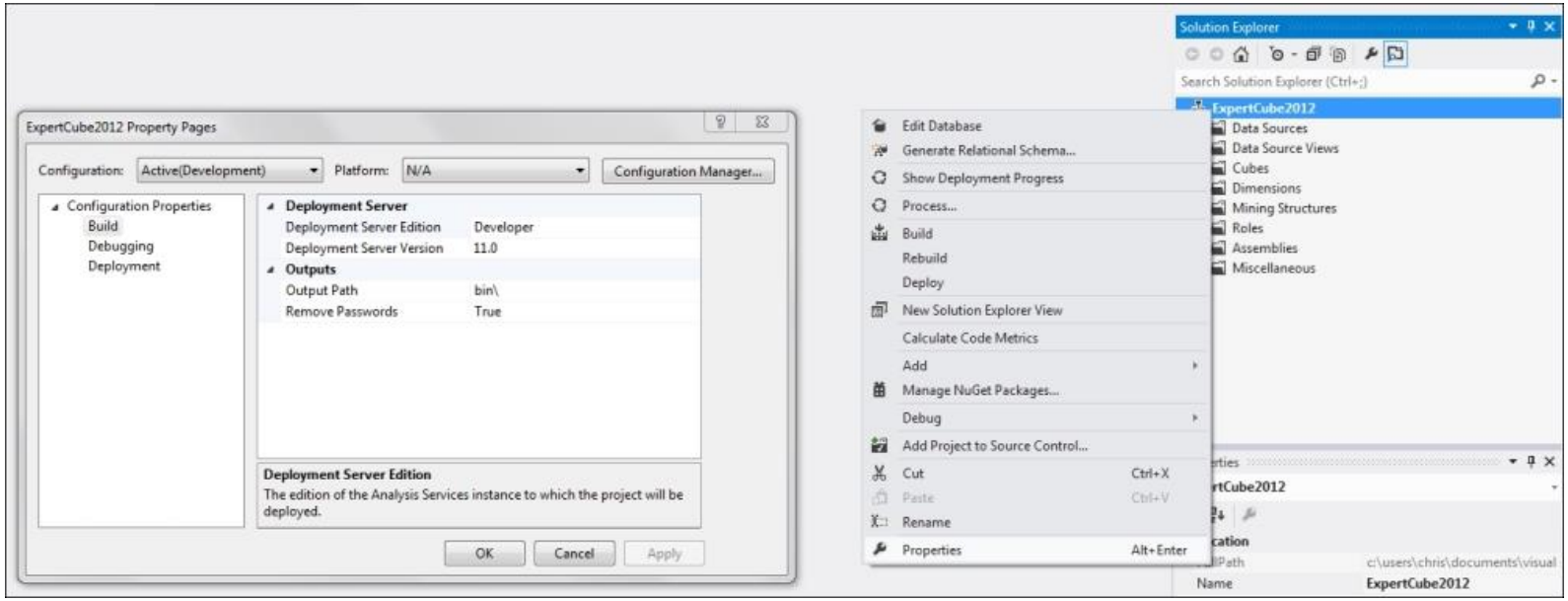
Analysis Services Multidimensional and Data Mining Project



Let’s start

Having prepared our relational source data, we're now ready to start designing a cube and some dimensions.

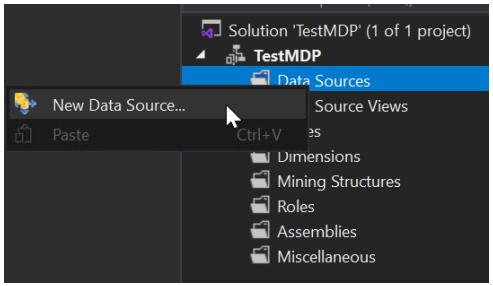
The first step towards creating a new cube is to create a new Analysis Services project in Visual Studio (SSDT)

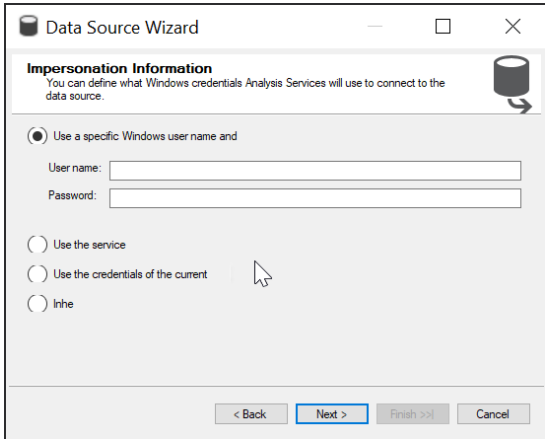


We need data

Once we've created a new project and configured it appropriately, the next step is to create a data source object.

- Even though you can create multiple data sources in a project, you probably shouldn't.
- Important – Analysis Services must also be given permission to access the data source
  - *On the Impersonation Information tab in the Data Source Designer dialog*
    - https://docs.microsoft.com/en-us/analysis-services/multidimensional-models/set-impersonation-options-ssas-multidimensional?view=sql-analysis-services-2022
      - Use a specific user name and password
      - Use the service account

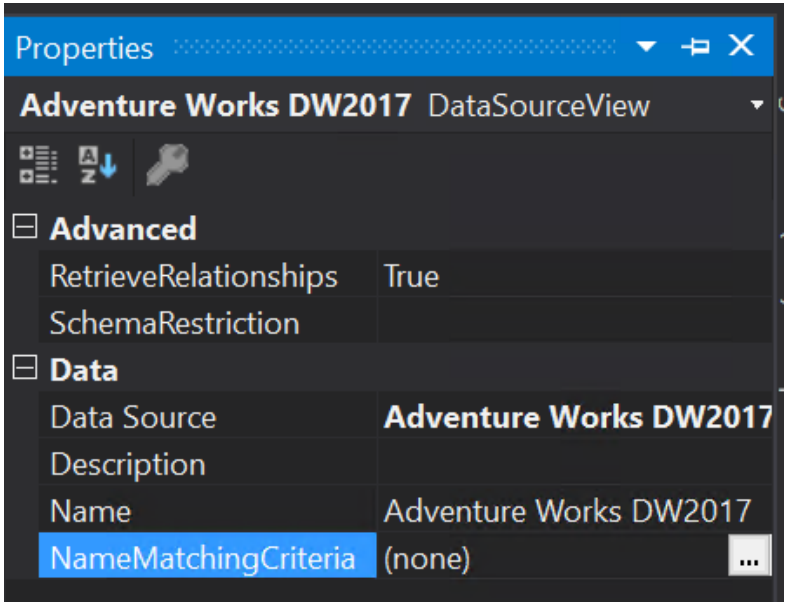




We need data

When you first create a new DSV, the easiest thing to do is to go through all of the steps of the wizard, but not to select any tables yet.

- You can then set some useful properties on the DSV, which will make the process of adding new tables and relationships much easier
  - *RetrieveRelationships*
    - True means that SSDT will add relationships between tables based on various criteria (foreign key relationships)
      - Depending on the value of the NameMatchingCriteria property, it may also use other criteria as well.
  - *SchemaRestriction*
    - Allows you to enter a comma-delimited list of schema names to restrict the list of tables that appear in the Add/Remove Tables dialog.
  - *NameMatchingCriteria*
    - True allows SSDT to try to guess relationships between tables by looking at column names.

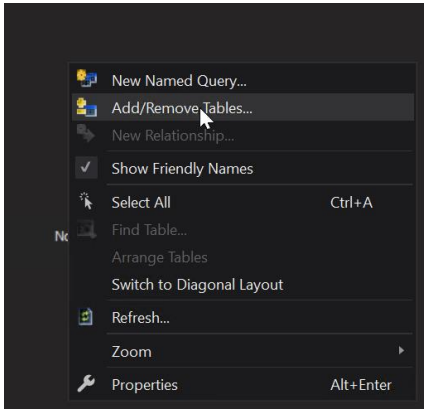


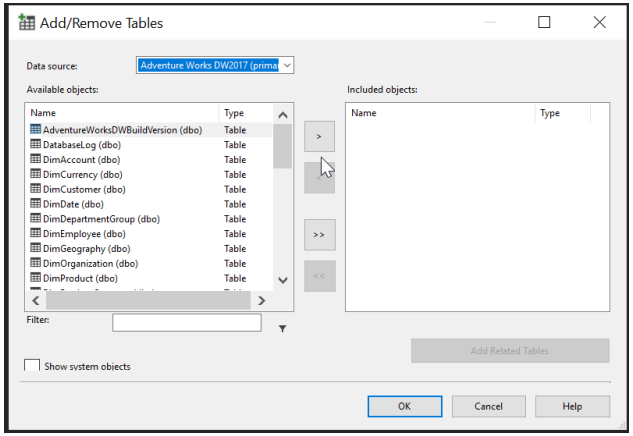
We need data

Now you can go ahead and right-click on the DSV design area and select
the Add/Remove Tables option

Further you can select any tables or views you need to use.

If you make changes in your relational data source, those changes won't be reflected in your DSV until you click on the Refresh Data Source View button or choose Refresh on the right-click menu.





Note

The Data Source View (DSV) is one of the places where we can create an interface between Analysis Services and the underlying relational model.

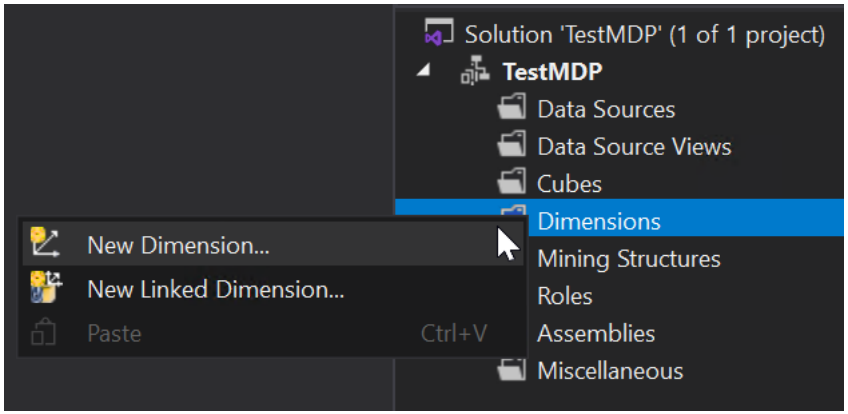
- In the DSV, we can specify joins between tables and we can create named queries and calculations to provide the equivalent of views and derived columns.
  - *It's very convenient for the cube developer to open up the DSV in SQL Server Data Tools and make these kind of changes.*
  - *It’s not a good idea to use the DSV for this purpose*

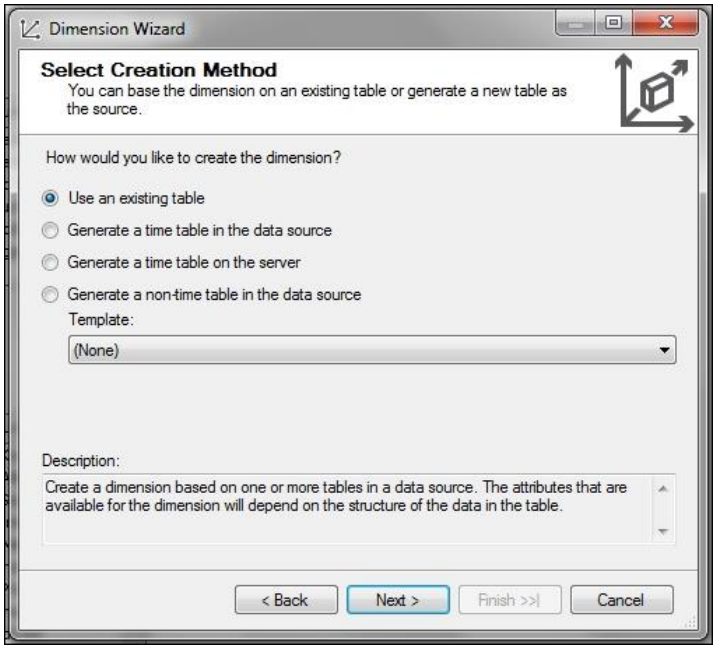
Building dimensions

Next, you can start building dimensions

Using the New Dimension wizard

Running the New Dimension wizard will give you the first draft of your dimension





Building dimensions

Select

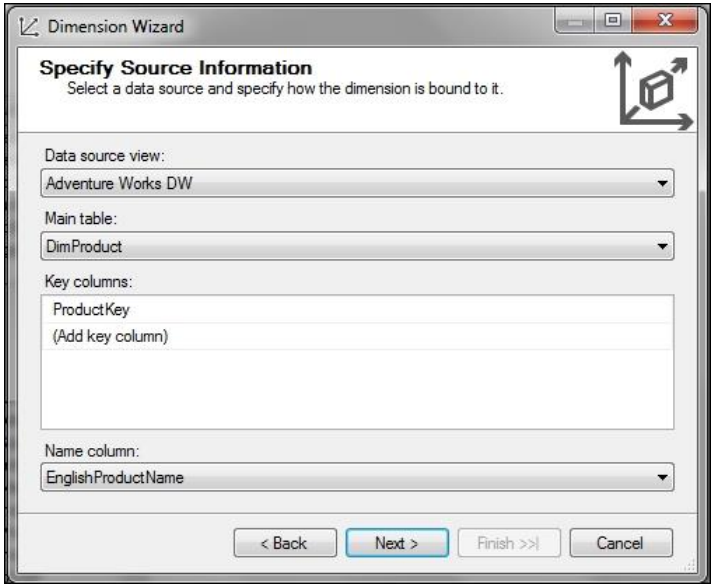
- Use an existing table option

In the next step of the wizard, you

- choose the main table for your dimension
  - *table or view that contains the lowest level of granularity of data for your dimension*
    - joins directly to a fact table in your DSV
- define the key attribute for the dimension
  - *attribute that represents the lowest level of granularity in the dimension*
    - to which all other attributes have a one-to-many relationship

Name column:

- Column that contains the name or description that the end user expects to see on screen when they browse the dimension.

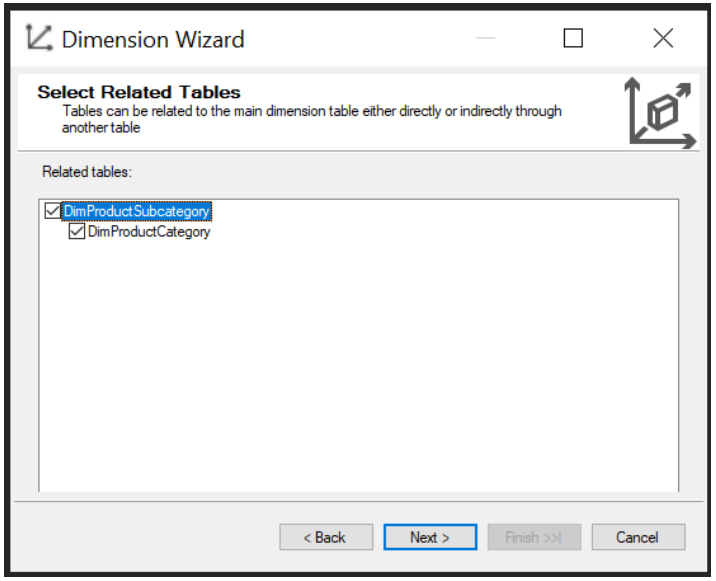


Building dimensions

Next you go to related tables

if you are building your dimension from a set of snowflaked tables you need to add additional tables,

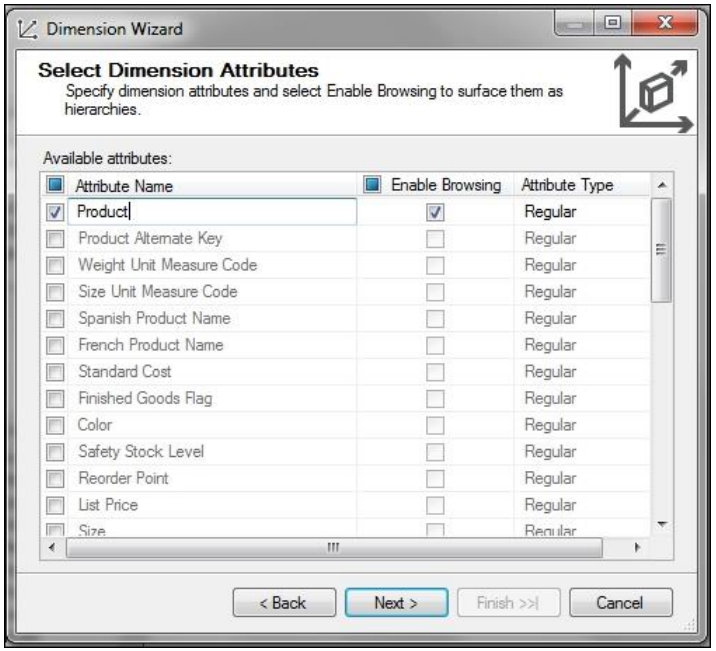
you have the option of selecting other tables you want to use to build this dimension.



Building dimensions

After this, you move onto the Select Dimension Attributes step

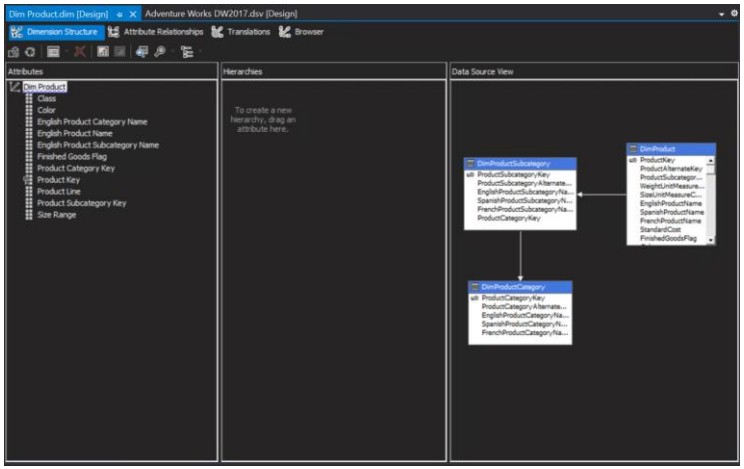
- you have the option of creating other new attributes on the dimension
  - *by checking any columns that you'd like to turn into attributes*
- you can also rename the attribute
- Check/uncheck the Enable Browsing option
  - *By default, an attribute hierarchy is created for every attribute in a dimension, and each hierarchy is available for dimensioning fact data.*
  - *This hierarchy consists of an "All" level and a detail level containing all members of the hierarchy.*
- Set the Attribute Type
  - *Important for Time, and Account*



Editing dimensions

Once you've completed the wizard, you'll arrive in the Dimension Editor for your new dimension.

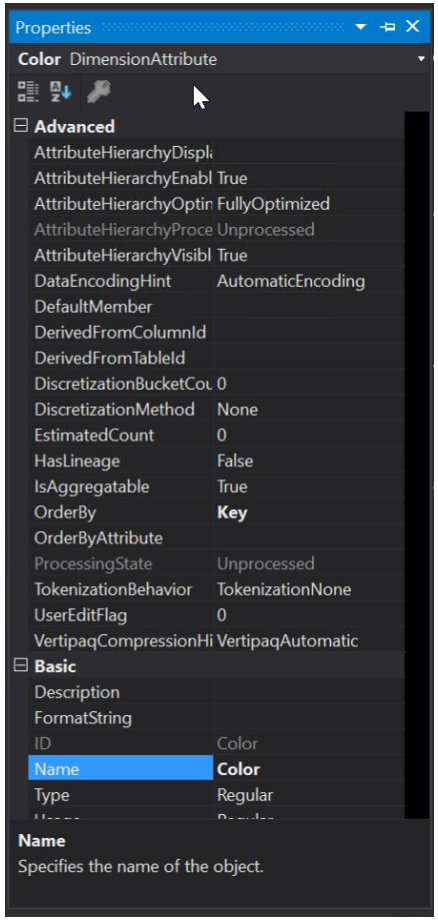
- to add more attributes you can simply drag columns
  - *from the Data Source View (right-hand side) into the Attributes pane (left-hand side)*
- to add user hierarchies you can simply drag and drop columns from attribute pane to hierarchies
  - *Top elemenet is the most general one*
  - *After creating a user hierarchy, you should definitely set the Visible property of its constituent attribute hierarchies to False*



Editing dimensions

Some important attribute properties

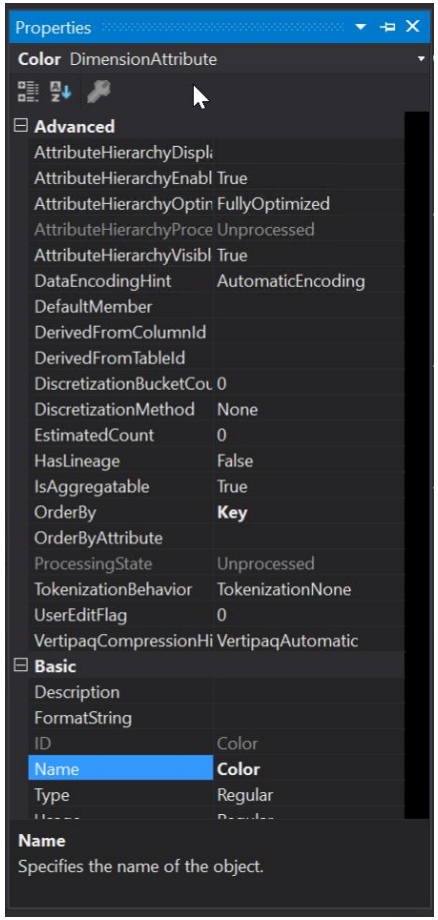
- KeyColumns, NameColumn
  - *column or columns that represent the key and the name of this attribute*
    - For attributes with a very large number of members, you should always try to use a column of the smallest possible numeric data type as the key to an attribute and set the name separately
- AttributeHierarchyEnabled:
  - *controls whether a hierarchy is built for this attribute,*
    - whether the user can actually see this attribute when they browse the cube and use it in their queries
- AttributeHierarchyOptimizedState:
  - *this property controls whether indexes are built for the enabled attribute hierarchy*
    - Setting this property to False can improve dimension processing times



Editing dimensions

Some important attribute properties

- OrderBy, OrderByAttribute
  - *order in which members appear in an attribute hierarchy can be important*
    - For example, you would expect days of the week to appear in the order of Sunday, Monday, Tuesday, and so on rather than in alphabetical order.
- Type
  - *Especially important for time dimension*
- Discretization method
  - *Ability to discretize an attribute – into discrete set of buckets*



Editing dimensions

- Proper ordering of attributes
  - Use OrderBy property of an attribute
    - *Name – alphabetically sorted*
    - *Key – based on key column value – in case of composite key, the first one is used*
    - *AttributeName / AttributeKey – based on a designated attribute (OrderByAttribute property)*
      - In order to make this work, the attribute used for sorting – color in our example - must be related to the attribute that needs to be sorted
      - If there is no attribute relationship, you cannot use the AttributeName value for the OrderBy property

Editing dimensions

Attribute relationships allow you to model one-to-many relationships between attributes.

Why? = Performance

they drastically improve Analysis Services' ability to build efficient indexes and make use of aggregations

It's very likely that there will be many different ways to model attribute relationships

It is important to find the best way from a performance point of view.

Default set of relationships

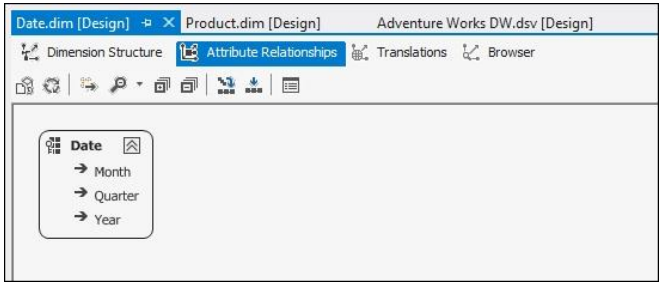
Every non-key attribute has a relationship defined either with the key attribute, or with the attribute built on the primary key of the source table in a snowflaked dimension.

Analysis Services knows that

a year is made up of many dates,

a quarter is made up of many dates,

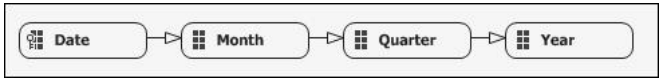
and a month is made up of many dates, but not that years are made up of quarters or that quarters are made up of months.

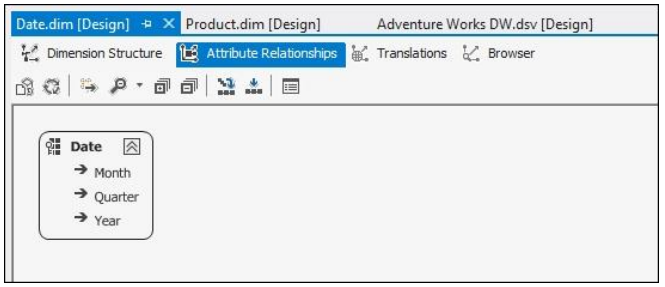


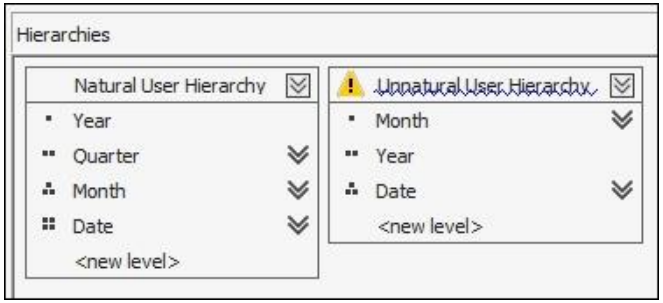
Editing dimensions

- Changing the attribute relationships
  - Drag and drop elements; use context menu
  - In general the longer the chain the better.
- Analysis Services understands transitory relationships
  - E.g. there is no need to define a relationship between Year and Date, because there is an indirect relationship via Quarter and Month.
- It is very important to understand your data before you set up any attribute relationships
  - if you set them up incorrectly, then it could result in Analysis Services returning incorrect data

Here's what the optimized set of relationships looks like in the Dimension Editor:







Editing dimensions

Check whether the attribute relationships you've defined actually reflect the data in your dimension table

a common mistake

- Year attribute has members 2001, 2002, 2003, and 2004;
- Quarter attribute has four members from Quarter 1 to Quarter 4
- Is there a one-to-many relationship between Year and Quarter?
  - *The answer is in fact no,*
    - because, for example, a quarter called Quarter 1 appears in every Year.

What you can and should do here is to modify your data

- so that you can build an attribute relationship

There are two ways to do this:

- By modifying the data in your dimension table (the best option)
  - *a quarter called Quarter 1 2001*
- By using a composite key in the KeyColumns property of the attribute
  - *use many attributes to uniquely identify a member*

Editing dimensions

RelationshipType property of an attribute relationship

- indicates whether the relationship between any two members on two related attributes is ever likely to change or not
  - *Rigid – do not change over time*
    - the date January 1st 2001 would always appear under the month January 2001
  - *Flexible – can change over time*
    - if a customer changes his/her residence the relationship between a Customer attribute and a City attribute may change over time

Setting RelationshipType to Rigid

- increases the chances that aggregations will not need to be rebuilt when a dimension undergoes a ProcessUpdate,

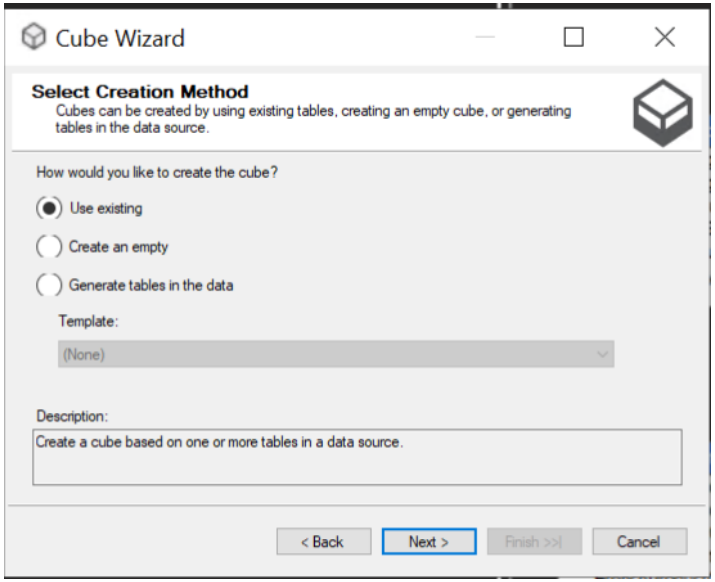
Creating cube

With some dimensions in place, the next step is to run the cube wizard to create the cube itself.

Using the New Cube wizard

- Again you can choose to
  - *Use an existing table*
  - *Create an Empty Cube*
  - *Generate Tables in the Data Source*

You should model your data properly in the data warehouse before you start building anything in Analysis Services.



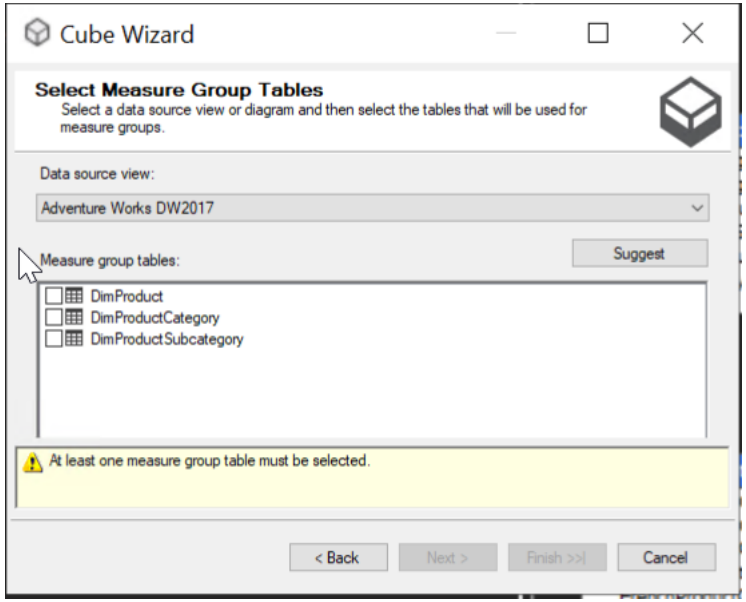
Creating cube

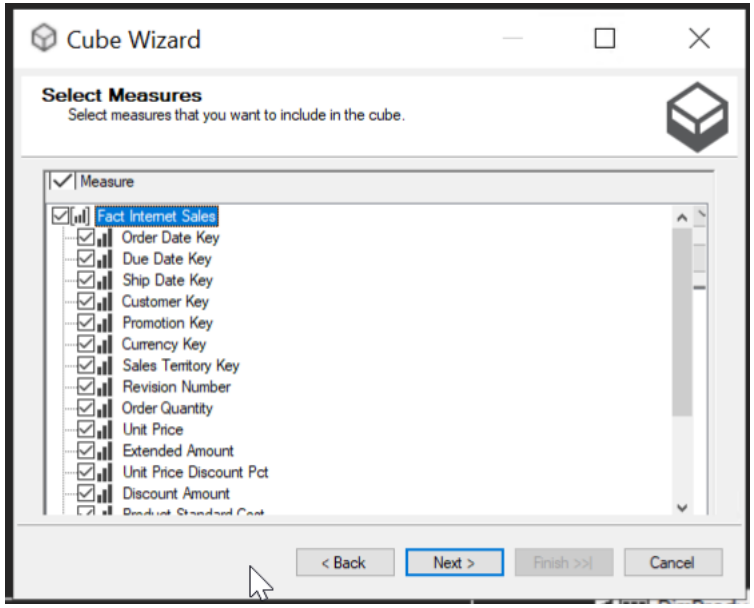
Next you select measures

On the Select Measure Group Tables step, just select the fact table

Then in the Select Measures step

select one or two columns from that fact table that represents commonly used measures, which can be aggregated by summation.





Creating cube

On the Select Existing Dimensions

- select all the dimensions that you've just built that join to the fact table you've chosen.

Next, you still have the possibility to add additional dimensions

- From data in the DSV

Finally, on the Completing the Wizard step, enter the name of the cube

As was the case when creating dimensions, it's worth putting some thought into the name of your cube.

- Try to make the name reflect the data the cube contains
  - *make sure the name is meaningful to the end users*
  - *keep it short*
    - you'll probably use it multiple times

Playing with measures

With our dimensions designed, we can now turn our attention to the cube itself.

Useful properties to set on measures

- FormatString
  - *Specifies how the row value of the measure gets formated when displayed*
- DisplayFolder
  - *Many cubes have a lot of measures on them, and as with dimension hierarchies, it's possible to group measures together into folders to make it easier for your users to find the one they want.*
  - *Subfolders are also possible*

Useful properties to set on measures

- AggregateFunction
  - *it controls how the measure aggregates up through each hierarchy in the cube*

Basic aggregation types

- Sum: the values for this measure will be summed up.
- MIN and MAX: return the minimum and maximum measure values.
- Count: counting the overall number of rows (from the fact table) or by counting non-null values from a specific measure column (depends on Source binding)
- DistinctCount: counts the number of distinct values in a column in your fact table
- None: no aggregation takes place on the measure at all.

Playing with measures

Continued

Semi-additive aggregation types are as follows:

AverageOfChildren

FirstChild

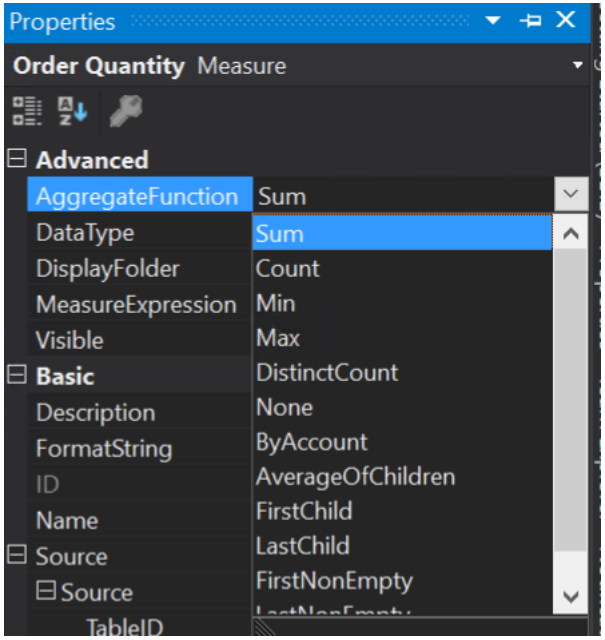
LastChild

FirstNonEmpty

LastNonEmpty

They behave the same as measures with aggregation type Sum on all dimensions except the Time dimension.

In order to get Analysis Services to recognize a Time dimension, you'll need to have set the dimension's Type property to Time in the Dimension Editor.



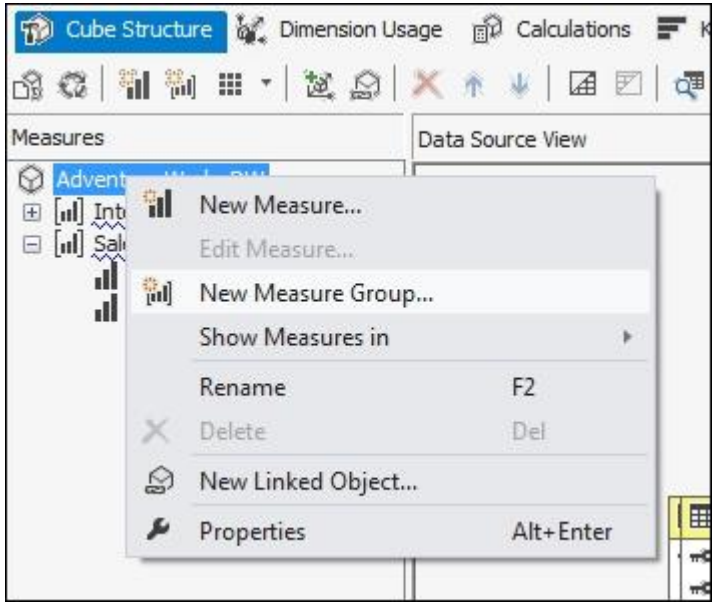
Playing with measures

All but the simplest data warehouses will contain multiple fact tables

- Analysis Services allows you to build a single cube on top of multiple fact tables through the creation of multiple measure groups.

To create a new measure group in the Cube Editor

- go to the Cube Structure tab and right-click on the cube name in the Measures pane and select New Measure Group.



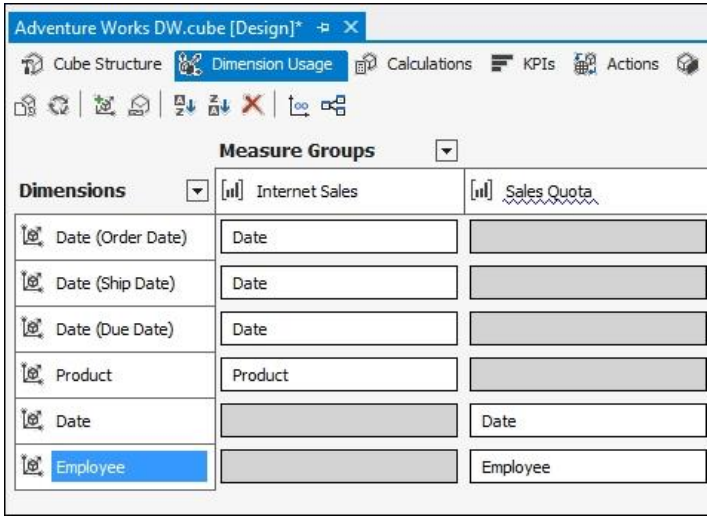
Playing with measures

Once you've created a new measure group

- SSDT will try to set up relationships between it and any existing dimensions in your cube
  - *based on the relationships you've defined in your DSV*
- You can check the relationships that have been created on the Dimension Usage tab of the Cube Editor

Measure groups don't always have to be created from fact tables.

- In many cases, it can be useful to build measure groups from dimension tables too.
- One common scenario where you might want to do this is when you want to create a measure that counts the number of elements in the dimension
  - *E.g. days in the currently selected time period*



Playing with measures

Calculated measure

Calculations enable you to define calculated members, named sets, and execute other script commands to extend the capabilities of an Analysis Services cube. For example, you can run a script command to define a subcube and then assign a calculation to the cells in the subcube.

In general, these are MDX expressions

Playing with measures

KPIs

Key Performance Indicator

a value which will determine how well an organization is achieving its objectives.

Organizations use KPIs at multiple levels to estimate their success at reaching objectives.

KPI in SSAS

Value: is the actual value of the KPI. This will be a numeric value. For example, this can be the Profit Margin.

Goal: every organization has a goal for this value. For example, the organization may look at the goal of achieving a five percent Profit Margin.

Also, sometimes they may have different values for different business areas. For example, depending on the product category or sales territory, the sales margin goal will differ.

Status: depending on the KPI value and the KPI goal, the KPI status can be defined. For an example, we can say that if the KPI value is greater than the goal it is great if it is not greater than the goal, but still greater than zero it is good and if less than zero or running at a loss it is bad.

This Great, Good or Bad can be displayed to the user by means of a graphical representation such as an arrow, traffic lights or a gauge.

Trend: optional parameter to depict the change of KPI's status. For example, you may have a great profit margin, but comparing with last year, it could be less. On the other hand, you might have a bad profit margin, but compared to last year it is improving.

Additional cube settings

Perspectives

perspectives provide the ability to create a data mart "like" subset of a SQL Server Analysis Services (SSAS) cube

acts and appears to many users as if it was an actual cube.

a predefined set of selected cube objects that are contained within the perspectives

Additional cube settings

Partitions

Partitioning is the concept where you divide your data from one logical unit into separate physical chunks.

This can have several advantages, such as improved performance or easier maintenance.

Typically, partitions on a measure group are based on a time column.

Partitioning on time also allows you to partition on different levels: old data can for example be put in yearly partitions, while new data can be partitioned at the month level or even at the day level.

SSAS

to start creating partitions, we need to go to the Partitions tab in the cube editor.

Two types – Table Binding (select all the data from a certain table, by using different views on top of a table for example) or Query Binding (specify the SQL query that fetches the data)

Additional cube settings

- Aggregations
  - Aggregations in SSAS offer an improvement of query performance and calculation times by "pre aggregating" sets of data.
    - *It is a trade off between creating a large number of aggregations (speed up query performance) versus increase of processing time and memory requirements.*
  - Aggregations are defined for a specific partition.
  - Different methods
    - *Based on size – using the required storage for aggregates*
    - *Based on the amount of space required to store and process the aggregation vs. the time it would take to query for the same values.*
    - *Based on usage – using query logs against SSAS*
    - *Manual - decide exactly what aggregations need to be created at any desired attribute or hierarchy level and combination of attributes.*
  - SSAS
    - *to start creating aggregations, we need to go to the Aggregations tab in the cube editor.*

Deploying

Having the cube defined you can build and deploy the cube into Analysis Services Server

- Use Build menu in the main Visual Studio menu bar and select
  - *Deploy <MyProjectName>.*

Deployment is actually a two-stage process:

- First the project is built.
  - *The end result is four files containing the XMLA representation of the objects in your project, and information on how the project should be deployed.*
    - You can find these files in the bin directory of your Visual Studio project directory.
- Then the project is deployed.
  - *This takes the XMLA created in the previous step, wraps it in an XMLA Alter command and then executes that command against your Analysis Services server.*
    - Executing this command either creates a new Analysis Services database if one did not exist before, or updates the existing database with any changes you've made.

Processing

Deployment

- Creates structures in the Analysis Services Server
  - *New/update multidimensional database*

Process

- In order to load the data you have to process the cube
  - *Using cube context menu and clicking on Process*

https://docs.microsoft.com/en-us/analysis-services/multidimensional-models/processing-options-and-settings-analysis-services?view=sql-analysis-services-2022

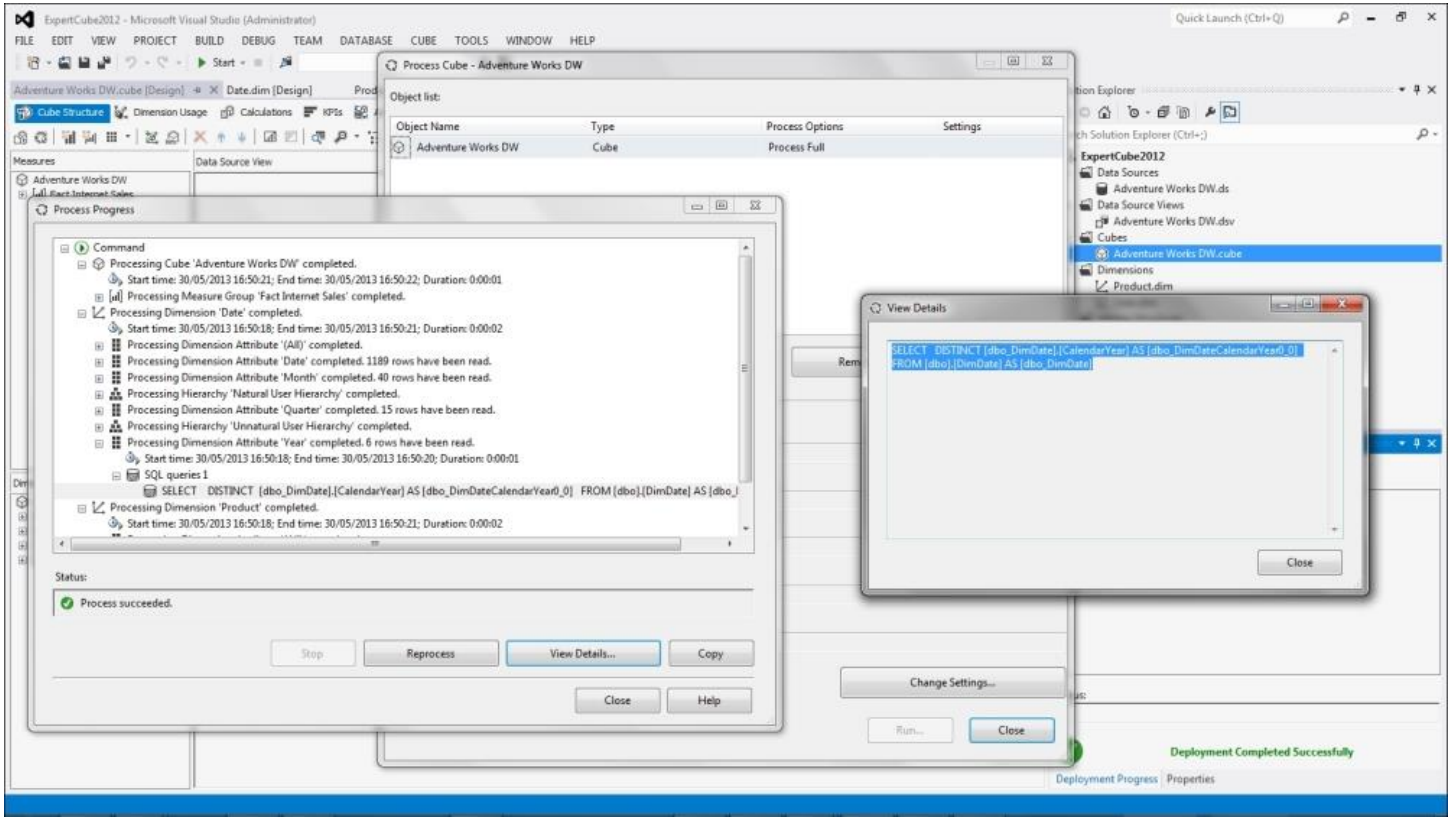
Cubes are made up of

- measure groups, which are made up of partitions,
- dimensions are made up of attributes,
- all of these objects have their own discrete processing operations

Analysis Services database is nothing more than a collection of objects:

- processing a database involves nothing more than processing all of the cubes and dimensions in the database.
  - *As a result, processing a database can kick off a lot of individual processing jobs (Analysis Services will try to do a lot of this processing in parallel)*

Processing

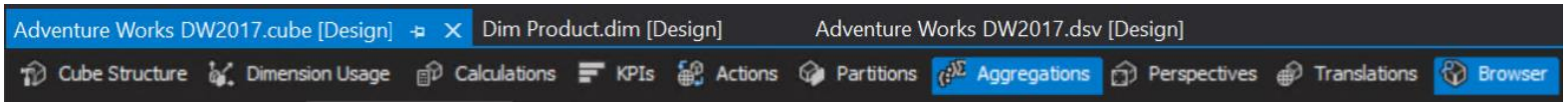

Process Cube - Adventure Works DW
Object list:
Object Name
Type
Process Options
Settings
Adventure Works DW
Cube
Process Full
Process Progress
Command
Processing Cube 'Adventure Works DW' completed.
Processing Measure Group 'Fact Internet Sales' completed.
Processing Dimension 'Date' completed.
Processing Dimension Attribute '(All)' completed.
Processing Dimension Attribute 'Date' completed. 1189 rows have been read.
Processing Dimension Attribute 'Month' completed. 40 rows have been read.
Processing Hierarchy 'Natural User Hierarchy' completed.
Processing Dimension Attribute 'Quarter' completed. 15 rows have been read.
Processing Hierarchy 'Unnatural User Hierarchy' completed.
Processing Dimension Attribute 'Year' completed. 6 rows have been read.
SQL queries 1
Processing Dimension 'Product' completed.
Status:
Process succeeded.
Stop
Reprocess
View Details...
Copy
Close
Help
View Details
Close
Change Settings...
Run...
Close
Deployment Completed Successfully
Deployment Progress
Properties

Browse

With processing complete, you can take a look at your cube for the first time, either in the Browser tab of the Cube Editor

Process recap

- Create new project
- Define data source
- Define data source view
- Define dimensions
- Define cube
- Build, deploy and process



Performance-specific options

Once you're sure that your cube design is as good as you can make it

Next you should look at two features of Analysis Services that are transparent to the end user, but have an important impact on performance and scalability:

- measure group partitioning
- measure group aggregations

Partitions

A partition is a data structure that holds some or all of the data held in a measure group.

When you create a measure group, by default that measure group contains a single partition that contains all of the data.

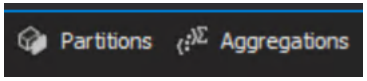
Analysis Services allow you to divide a measure group into multiple partitions

Aggregations

An aggregation is simply a pre-summarized data set

similar to the result of a SQL SELECT statement with a GROUP BY clause, that Analysis Services can use when answering queries

The advantage of having aggregations built in your cube is that it reduces the amount of aggregation that the Analysis Services Storage Engine has to do at query time



Performance-specific options

Partitions

brings two important benefits: better manageability and better performance.

within the same measure group can have different storage modes and different aggregation designs

more importantly they can be processed independently

E.g., when new data is loaded into a fact table, you can process only the partitions that should contain the new data.

E.g., if you need to remove old or incorrect data from your cube, you can delete or reprocess a partition without affecting the rest of the measure group.

can also improve query performance significantly.

Analysis Services can process multiple partitions in parallel

Building partitions

You can view, create, and delete partitions on the Partitions tab of the Cube Editor.

When you run the 'New Partition' wizard or edit the Source property of an existing partition, you'll see you have two options for controlling what data is used in the partition:

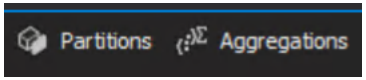
Table Binding:

This means that the partition contains all of the data in a table or view in your relational data source, or a named query defined in your DSV.

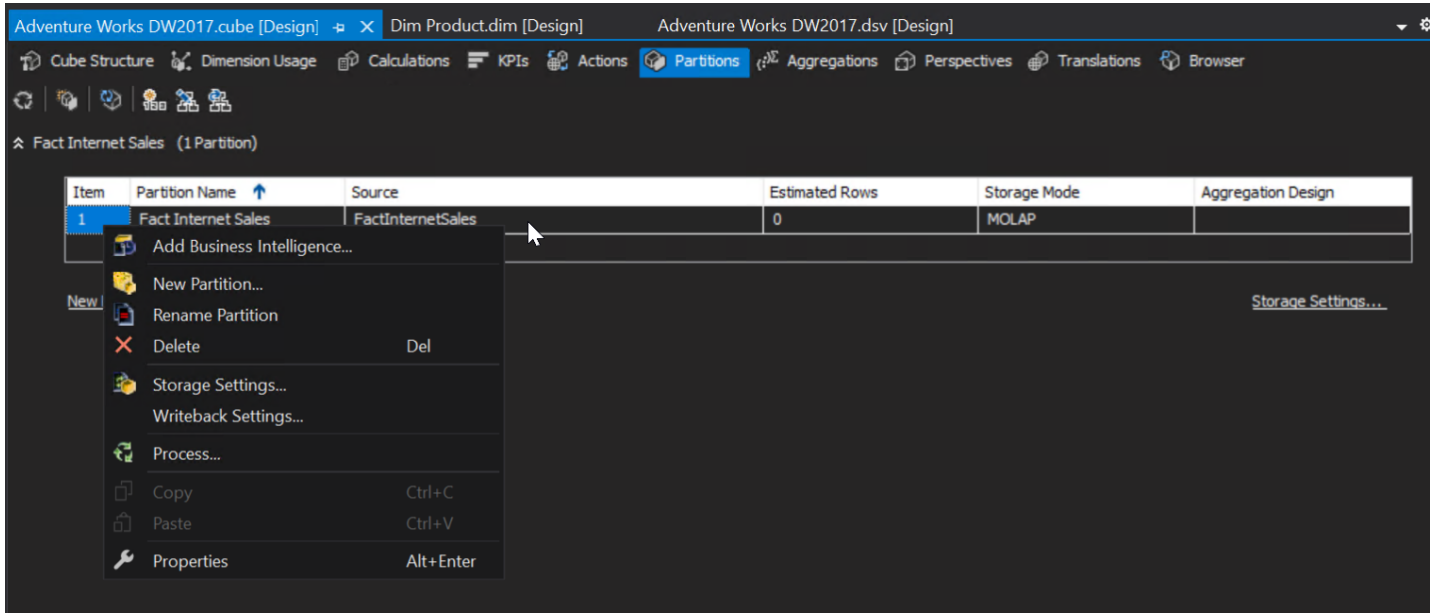
Query Binding:

This allows you to specify a SQL SELECT statement to filter the rows you want from a table;

SSDT will automatically generate part of the SELECT statement for you, and all you'll need to do is supply the WHERE clause.



Performance-specific options



Performance-specific options

Remote Partitions

On the Processing and Storage Locations step of the wizard you have the chance to create the partition on a remote server instance

Storage mode

Affects the query and processing performance, storage requirements, and storage locations of the partition and its parent measure group and cube

A partition can use one of three basic storage modes:

Multidimensional OLAP (MOLAP)

Relational OLAP (ROLAP)

Hybrid OLAP (HOLAP)

How should we split the data in our partitions?

We need to find some kind of balance between the manageability and performance aspects of partitioning

we need to split our data so that we do as little processing as possible, but also so the division of data means that as few partitions are scanned as possible by our users' queries.

Luckily, if we partition by our Time dimension we can usually meet both needs very well

it's almost always the case that when measure groups are partitioned they are partitioned by Time

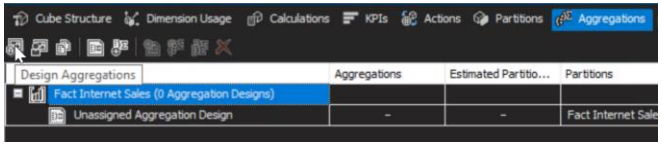
Performance-specific options

The first stage in creating an aggregation design should be to create a core set of aggregations that will be generally useful for most queries run against your cube

- This should take place towards the end of the development cycle when you're sure that your cube and dimension designs are unlikely to change much
- Removing unnecessary attributes, setting AttributeHierarchyEnabled to False where possible, building optimal attribute relationships and building user hierarchies will all make the aggregation design process faster, easier and more effective.

To build this an initial set of aggregations you'll be running the Aggregation Design Wizard.

- This is available through Design Aggregations button on the toolbar of the Aggregations tab of the Cube Editor.



Performance-specific options

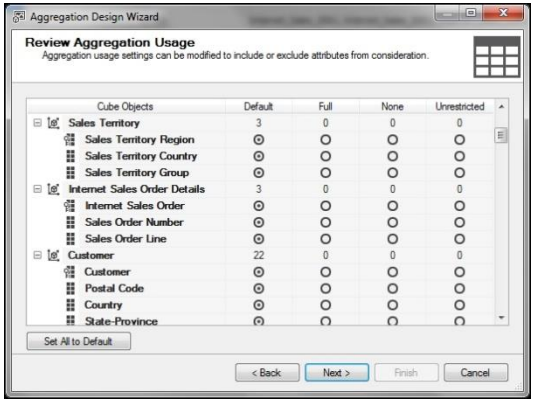
The Design Aggregation wizard will analyze the structure of your cube and dimensions

look at various property values you've set and try to come up with a set of aggregations that it thinks should be useful.

The one key piece of information it doesn't have at this point is what queries you're running against the cube,

so some of the aggregations it designs may not prove to be useful in the long run,

still running the wizard is extremely useful for creating a first draft of your aggregation designs



The first step is the Review Aggregation Usage:

controls how dimension attributes are treated in the aggregation design process

can be set to the following values:

Full: the attribute or an attribute at a lower granularity directly related to it by an attribute relationship will be included in every single aggregation the wizard builds.

It is recommended that you use this value sparingly, for at most one or two attributes in your cube because it can significantly reduce the number of aggregations that get built.

You should set it for attributes that will almost always get used in queries.

None: the attribute will not be included in any aggregation that the wizard designs.

Don't be afraid of using this value for any attributes that you don't think will be used often in your queries; it can be a useful way of ensuring that the attributes that are used often get good aggregation coverage.

Performance-specific options

Continued

Unrestricted: the attribute may be included in the aggregations designed, depending on whether the algorithm used by the wizard considers it to be useful or not.

Default: applies a complex set of rules, which are:

The granularity attribute (usually the key attribute, unless you specified otherwise in the dimension usage tab) is treated as Unrestricted.

All attributes on dimensions involved in many-to-many relationships, unmaterialized referenced relationships, and data mining dimensions are treated as None.

All attributes that are used as levels in natural user hierarchies are treated as Unrestricted.

Attributes with IsAggregatable set to False are treated as Full.

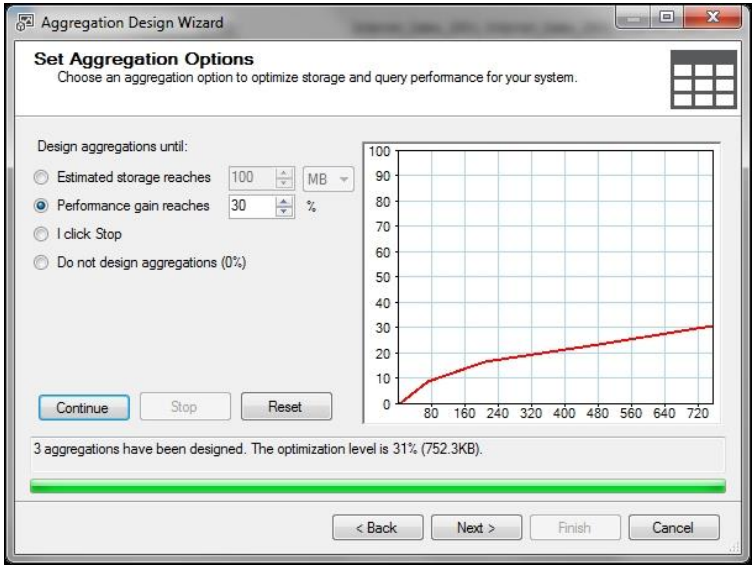
All other attributes are treated as None.

Note that attributes with AttributeHierarchyEnabled set to False will have no aggregations designed for them anyway.

Performance-specific options

The next step in the wizard asks you to verify the number of EstimatedRows

On the Set Aggregation Options step you finally reach the point where some aggregations can be built



You can select one of possible strategies:

Estimated Storage Reaches

build aggregations to fill a given amount of disk space.

Performance Gain Reaches:

the most useful option.

It does not mean that all queries will run n percent faster, nor does it mean that a query that hits an aggregation directly will run n percent faster

Setting this property to 30%, the default and recommended value, will build the aggregations that give you 30% of this possible performance gain – not 30% of the possible aggregations, usually a much smaller number

Performance-specific options

Continued

I Click Stop:

System is building aggregations until you click on the Stop button.

Designing aggregations can take a very long time, especially on more complex cubes because

Do Not Design Aggregations:

To skip designing aggregations automatically

Usage-Based Optimization

We now have some aggregations designed, but the chances are that despite our best efforts many of them will not prove to be useful.

it allows us to log the requests for data that Analysis Services makes when a query is run and then feed this information into the aggregation design process.

Performance-specific options

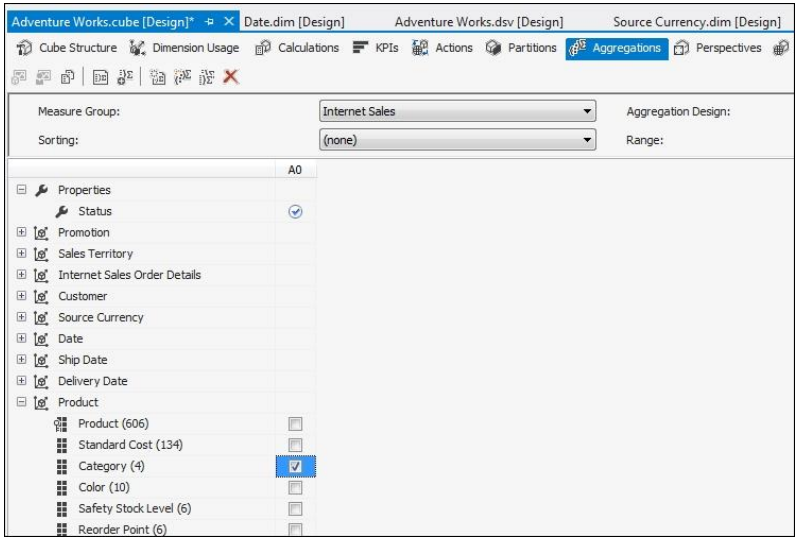
Building aggregations manually

In SSDT, to view and edit aggregations, you need to go to the Aggregations tab in the cube editor.

On the Standard view you only see a list of partitions and which aggregation designs they have associated with them;

if you switch to the Advanced view by pressing the appropriate button on the toolbar, you can view the aggregations in each aggregation design for each measure group.

If you right-click in the area where the aggregations are displayed you can also create a new aggregation and once you've done that you can specify the granularity of the aggregation by checking and unchecking the attributes on each dimension.



Bibliography

SOURCES

Kimball R., Ross M.

The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd Edition

Wiley Publishing, 2013

Hughes S., Jorgensen A.,

Hands-On SQL Server 2019 Analysis Services

Packt Publishing, 2020

Webb C., Ferrari A., Russo M.

Expert Cube Development with SSAS Multidimensional Models

Packt Publishing, 2014

Dewald B., Turley P., Hughes S.

SQL Server Analysis Services 2012 Cube Development Cookbook

Packt Publishing, 2013